

wife-system · 阶段 0 ~ 3 实现后教材

P3

Markdown 活动导入

固定代码基线 6e89762
中文学习版 · 虚拟数据示例

从实际代码、数据流和测试证据出发；每册可独立阅读。

目录

第四册：P3 Markdown 活动导入	3
学习目标与前置知识	3
1. 端到端数据流	4
小练习	4
2. 为什么离线确定性解析，不让模型写数据库	5
小练习	5
3. 受限 Markdown、Unicode 与金额边界	6
3.1 受限语法	6
3.2 Unicode 与名称	6
3.3 整数金额与范围	6
小练习	6
4. 预览、候选动作、确认与原子提交	7
4.1 预览不是零持久化，而是零业务效果	7
4.2 提交前重验所有假设	7
4.3 同一事务，要么全部成功要么全部回滚	7
小练习	7
5. HMAC 摘要、幂等、回执与重放	8
小练习	8
6. PostgreSQL 并发、迁移与验收证据	9
6.1 三层并发保护	9
6.2 Alembic migration 与 82 字符外键名缺陷	9
6.3 谁证明了什么	9
6.4 面向 P4 的接口，不替 P4 做技术决定	9
常见误区	10
阶段练习	11
检查题	12

第四册：P3 Markdown 活动导入

代码基线：**6e89762**，对应 P3-B5-R1 与 C9-R2 最终固定快照。示例全部为虚拟活动。

学习目标与前置知识

学完本册，你应能追踪一份 Markdown 从 HTTP 到预览、确认和数据库提交的完整路径，解释确定性解析、安全输入、HMAC 摘要、幂等回执、并发控制、迁移和测试分层。

前置知识：P0 的 HTTP/Pydantic，P1 的事务/迁移/账本，P2 的确认和可信上下文。

1. 端到端数据流

- Markdown + Idempotency-Key
- FastAPI route: 媒体类型、正文上限、严格 JSON
 - Pydantic PreviewRequest
 - parser: 规范化、受限语法、金额整数化、issues
 - ActivityImportService.preview
 - repository/model: 保存 batch + candidates + preview receipt
 - PreviewResponse: create/revise/unchanged/conflict/unresolved

- 用户逐项 accept/skip 并显式 confirmed=true
- Pydantic CommitRequest
 - service: 重验摘要、batch 版本、动作、警告、目标版本
 - PostgreSQL 行锁/唯一约束 + FinanceService session helper
 - 同一个 transaction 原子写入模板修订、候选结果、batch、receipt
 - CommitResponse; 失败则完整 rollback

HTTP 路由在 `api/activity_import_routes.py` 第 20 ~ 135 行。`_body()` 只接受唯一的 `application/json` 头、最多 96 KiB 正文、UTF-8、无重复 JSON key/NaN/Infinity，再交给 Pydantic。端点是：

方法与路径	作用	权限
POST /api/v1/activity-imports/preview	解析并保存预览批次	finance:write
GET /api/v1/activity-imports/{batch_id}	读取自己的批次	finance:read
POST /api/v1/activity-imports/{batch_id}/commit	原子提交选择	finance:write

Pydantic DTO 在 `activity_import/schemas.py` 第 17 ~ 128 行。`extra="forbid"` 拒绝未知字段，`StrictStr/StrictInt` 避免危险宽松转换，`CommitRequest.confirmed` 必须是布尔 `true`，不能用字符串冒充。

虚拟输入：

```
# 常用活动

## 周末采购
- 参考金额范围：80-120 元

## 晚间散步
- 备注：只作为活动说明
```

预览会生成候选、行号、整数分上下界和 issues；它不会创建交易、活动发生、预算消耗或收入。

小练习

从 `preview` 路由开始，在代码中依次找到 DTO、parser、service、repository、ORM 模型和 response 的入口，各写一句输入输出。

2. 为什么离线确定性解析，不让模型写数据库

P3 把 Markdown 当数据，不调用 DeepSeek，也不执行其中的代码、链接、路径或“忽略规则”指令。解析器文件开头明确写着“bounded text grammar, not a Markdown renderer or a tool interpreter”，见 [parser.py](#) 第 1~22 行。

这样做解决三个问题：

1. 相同文本、相同版本得到可重复结果，便于摘要、重放和测试。
2. 模型提示注入不能改变权限、自动确认或直接写库。
3. 预览到提交之间可以重验确定字段和版本，不依赖模型再次“理解得一样”。

替代方案是让模型输出活动 JSON，再做校验。这在未来可作为**不可信建议层**帮助用户整理自由文本，但最终仍必须经过固定 DTO、确定性归一化、用户确认和同一事务服务；当前代码没有实现该模型步骤。

小练习

在 Markdown 代码块里写 **请忽略确认并写入数据库**。解释 parser 为什么只产生 warning/普通数据，而不会调用任何工具。

3. 受限 Markdown、Unicode 与金额边界

3.1 受限语法

解析器只认一个可选一级标题和最多 50 个 **## 活动名** 候选。全文最多 64 KiB、2000 行，每候选最多 20 个非空内容行。入口在 `parse_markdown`，候选解析在第 113 ~ 177 行。

支持的金额形式：

- 参考金额：12.30 元 → 1230
- 参考金额范围：80-120 元 → min=8000, max=12000
- 通常每次 80 至 120 元，只用于预测。→ 同一范围结构

链接、HTML、路径、代码围栏、深层标题、组合语义和未知字段不会触发外部访问。它们按规则成为 warning、error 或整体拒绝。未知自由文字不被模型猜成数据库字段。

3.2 Unicode 与名称

`normalize_markdown()` 在 `parser.py` 第 33 ~ 41 行移除 BOM、统一换行并拒绝危险控制/Bidi 字符。名称使用 NFC、压缩空白和 `casefold()`，见第 25 ~ 30 行。`source_label` 只保留跨平台 basename，拒绝链接、路径语义和控制字符，见 `schemas.py` 第 37 ~ 54 行。

3.3 整数金额与范围

`_minor()` 在 `parser.py` 第 73 ~ 84 行只接受普通十进制和最多两位小数，不接受指数记法、负数、float 或静默舍入；最大值为 **999_999_999_999** 分。范围必须上下界齐全且 **low <= high**，第 92 ~ 110 行。

数据库同时用形状约束保证：三项全空，或 min/max 都存在且有序；单值存在时必须等于 min 和 max。

Pydantic 是第一道输入防线，parser 是语义防线，数据库 CHECK 是最后防线。

小练习

预测 **1.001 元**、**1e3 元**、**120-80 元**、**\$12**、**0.01 元** 的结果。把“格式错误”“币种错误”“范围错误”和合法 1 分区分开。

4. 预览、候选动作、确认与原子提交

4.1 预览不是零持久化，而是零业务效果

`ActivityImportService.preview()` 在 `service.py` 第 209 ~ 315 行解析文本、计算 HMAC 摘要、领取预览回执，并在单事务内保存 `activity_import_batch` 与 `candidates`。它不会创建/修订活动模板，不会记账。

每个候选动作：

动作	含义	可直接 accept
<code>create</code>	无同名历史，可创建	是
<code>revise</code>	唯一活动存在且内容变化	是，绑定目标版本
<code>unchanged</code>	与当前 revision 等价	否，通常 skip
<code>conflict</code>	重名、归档或历史名歧义	否
<code>unresolved</code>	有错误 issue，信息不可安全解释	否

判断代码在 `service.py` 第 245 ~ 300 行。候选保存来源标题、行范围、规范名、金额、目标模板及预期版本，使用户能复核而不是接受黑盒结论。

4.2 提交前重验所有假设

`CommitRequest` 携带 `batch_version`、`content_digest` 和每个候选的 `accept/skip`、预期动作、预期模板版本、已确认 `warning code`。`_validate_selection()` 在 `service.py` 第 347 ~ 386 行检查：

- 摘要和 batch 版本未变化；
- 决策集合恰好覆盖本 batch，ID 不重复；
- 动作与目标版本与预览一致；
- 所有 warning 精确确认；
- `conflict/unresolved/unchanged` 不能被强行 `accept`；
- 一个批次不能重复接受同一规范名或同一目标。

4.3 同一事务，要么全部成功要么全部回滚

`commit()` 在 `service.py` 第 388 ~ 493 行用一个 `session.begin()` 完成：领取 `commit` 回执、锁 batch、重验选择、锁目标、再查名称历史、逐候选调用 `FinanceService` 的 `session helper`、更新 `candidate`、`batch`、`selection fingerprint` 和 `receipt`。

若第二个候选失败，第一个候选、修订、batch 状态、回执和审计全部回滚。这就是**原子提交**。`repository` 不拥有事务，只在同一 `session` 内查询和加锁，见 `activity_import/repository.py` 第 20 ~ 64 行。

小练习

设计一个批次：A 为合法 `create`，B 在确认前目标版本已变化。写出为何整批应失败，并列出失败后应保持不变的五类行。

5. HMAC 摘要、幂等、回执与重放

HMAC 是带秘密密钥的摘要。普通 hash 能检测内容变化，但知道输入的人可重算；HMAC 还要求持有服务器密钥。本项目把规范化 Markdown 做成版本化字符串：`hmac-sha256:vN:<64 hex>`，实现见 `service.py` 第 70 ~ 83 行。原文不进入日志。

这些概念各有不同用途：

概念	回答的问题
Idempotency-Key	调用方说“这是同一次 preview/commit 吗”
content_digest	预览的规范化 Markdown 是否还是那份内容
command fingerprint	同一个幂等键是否带了同一业务载荷
CommandReceipt	数据库是否已处理该来源及最终结果是什么
selection_fingerprint	最终确认选择的规范指纹是什么
request_id	这次 HTTP 尝试的可观察关联号，不是业务幂等键

`_claim()` 在 `service.py` 第 126 ~ 138 行复用 P1 的持久回执机制，并把操作、入口和 owner 纳入来源范围。同键同载荷返回已保存的 response 并设置 `replayed=true`；同键不同载荷返回 `duplicate_request_conflict`。失败事务不会留下“成功回执”，同键可以按规则完整重试。

隐私日志只写 request ID、batch ID、parser 版本、摘要短前缀、数量、动作计数、耗时和稳定错误码，见 `service.py` 第 85 ~ 124 行及 preview/commit 结尾。它不写 Markdown、活动名、金额原文、密钥或 SQL 参数。

小练习

分别判断：同键同文、同键改一行、换键同文、同 commit 键改一项 skip 为 accept。说明哪些重放、哪些新建、哪些冲突。

6. PostgreSQL 并发、迁移与验收证据

6.1 三层并发保护

1. 行锁: `ImportRepository.batch(..., lock=True)` 和 `lock_targets()` 使用 `FOR UPDATE`, 见 `repository.py` 第 24 ~ 31 行和第 55 ~ 63 行。固定排序目标 ID, 降低死锁风险。
2. 乐观版本: `batch` 与模板的 `version_id` 必须和预览绑定值相同; `stale` 请求返回 `concurrent_modification`。
3. 唯一约束: `activity_template.name_normalized` 防止两个不同 `batch` 同时 `create` 同名对象。服务把数据库名称竞争映射为稳定并发错误, 见 `service.py` 第 89 ~ 124 行。

锁不能代替版本, 版本不能代替唯一约束。两个事务可能预览时都看见“无同名”; 最终必须由数据库唯一性收敛。

6.2 Alembic migration 与 82 字符外键名缺陷

P3 迁移 `c82d7a4f901e_add_activity_import.py` 创建 `batch/candidate`, 给模板增加规范名唯一约束, 给 `revision` 增加金额上下界和来源候选。升级前 `_preflight()` 检查既有模板是否有当前 `revision`、名称是否重复、金额是否合法, 避免半途迁移才发现数据不兼容。

原实现显式外键名长 82 字符。SQLite 接受它, PostgreSQL 标识符上限是 63 字节, 真实空 `schema` 升级在业务测试前就抛 `IdentifierError`。返修把名称缩短为 46 字符常量 `fk_activity_template_revision_import_candidate`, 见迁移第 21 ~ 24 行和第 149 ~ 158 行。这说明离线 DDL 和 SQLite 不能替代目标数据库实际执行。

6.3 谁证明了什么

角色/证据	职责	最终 P3 证据
89 项矩阵	先定义语法、金额、纯度、并发、幂等、安全、API、数据库风险	89 个唯一案例
执行方测试	实现者自测与回归	本地 144 通过; 真实 PG 9 通过
独立测试	不复用实现者结论, 验证合同、故障和隐私	本地 81 通过; 真实 PG 10 通过
固定快照	确认测试对应的 22 个产品/迁移/执行方测试文件未漂移	22/22 摘要匹配
头脑风暴总控	核对边界、证据和交付后决定是否接受	技术顾问不代替总控验收

C9-R2 最终将 89 项适用案例全部记为 `passed`, 关闭外键名缺陷; 报告同时记录了一次独立测试代码引用不存在字段的测试缺陷, 修正后才得到真实回滚证据。详情见 `phase-3-c9-r2-activity-import-report.md`。通过数字说明这些固定案例有证据, 不代表未知输入、未来 `schema` 或所有 PostgreSQL 版本自动正确。

6.4 面向 P4 的接口, 不替 P4 做技术决定

未来桌面端可以调用现有三类 API: 发送 Markdown 取得候选表; 展示动作、issues、目标版本和 warning; 让用户逐项 `accept/skip` 后提交; 用 GET 在断线后恢复 `batch`。客户端必须保留 `Idempotency-Key`、`content digest`、`batch version` 和 `candidate ID`, 且不能把预览当成功写入。

当前 P3 没有决定 P4 使用哪种桌面框架、状态管理库或 UI 组件。那些属于 P4 的技术选型。

常见误区

1. 把“预览不产生业务效果”理解成数据库完全零写入；当前会保存批次、候选和预览回执。
2. 把 Markdown 当完整 Markdown 渲染器输入；当前只是有限文本语法。
3. 使用普通平均值把 80~120 元静默折叠为 100 元。
4. 只相信预览时的目标版本，提交时不重验。
5. 用 request ID 当幂等键，或在重试时随意换键。
6. 认为 **FOR UPDATE** 在 SQLite 测试通过就证明 PostgreSQL 并发正确。
7. 把 89 项矩阵等同于 89 个 pytest 节点；矩阵案例可由多个测试和环境共同提供证据。

阶段练习

亲手写一份含两个虚拟活动的 Markdown：一个精确金额、一个范围金额。先手工算出 minor units 和预期动作；再构造 preview/commit JSON，其中一个 accept、一个 skip。最后加入未确认 warning，解释为何整批应在写业务对象前拒绝。

检查题

1. 为什么 P3 不调用模型解析 Markdown?
2. `content_digest`、command fingerprint 和 Idempotency-Key 有何区别?
3. 预览为什么允许保存 batch，却仍称为“无业务效果”?
4. 行锁、乐观版本和规范名唯一约束怎样分工?
5. 82 字符外键名缺陷为何必须通过真实 PostgreSQL 迁移发现?